

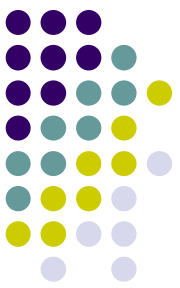
Command Pattern

Architecture of Real-Time Systems
SE 456

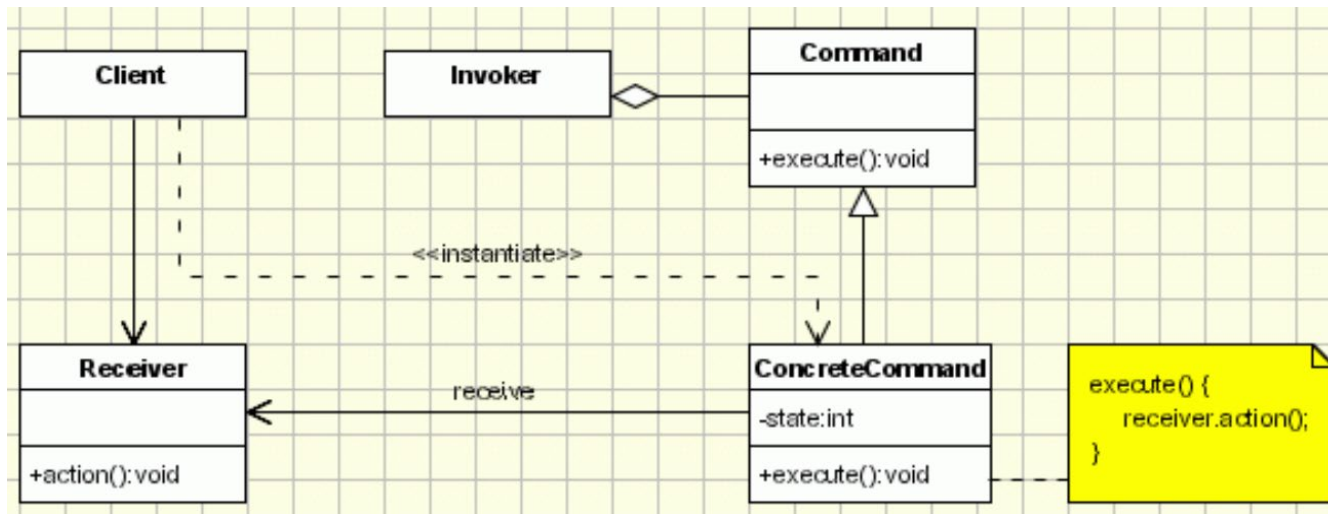
Ed Keenan

27 January 2021

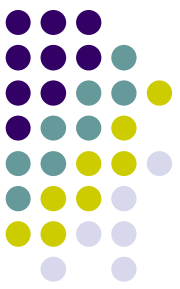
13.0.8.3.19 Mayan Long Count



Command



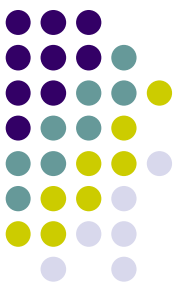
- Intent
 - Encapsulate a request in an object
 - Allows the parameterization of clients with different requests
- Remark
 - Command design pattern encapsulates commands (method calls) in objects allowing us to issue requests without knowing the requested operation or the requesting object.
 - Command design pattern provides the options to queue commands, undo/redo actions and other manipulations.



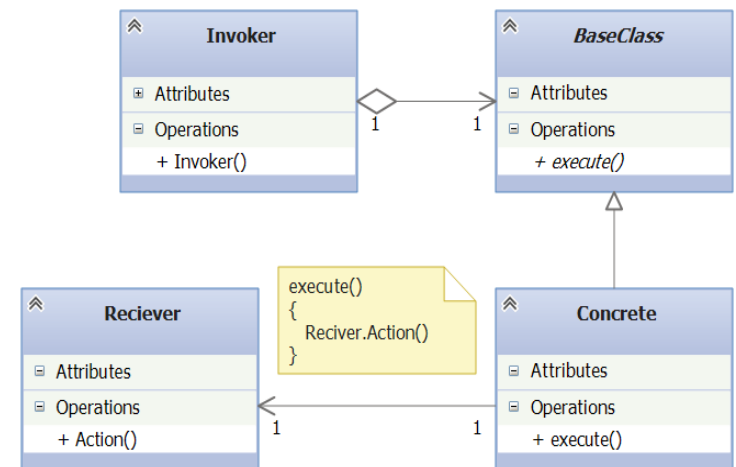
Command

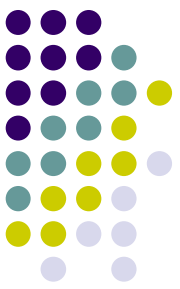
- Command Pattern **encapsulates a request as an object**
 - You can pass the request to objects
 - queue or log requests
 - support undoable operations
- Encapsulate method invocation.
 - object invoking the computation doesn't need to worry about how to do things

Command



- **Command object** has a reference to the receiver object
 - Invokes a method of the receiver
- **Receiver** then does the work.
- **Invoker** makes a request of a Command object by calling its execute() method, which invokes those actions on the receiver.
 - *Any command object can be passed to the same invoker object.*
- **Client** contains the decision making about which commands to execute at which points.

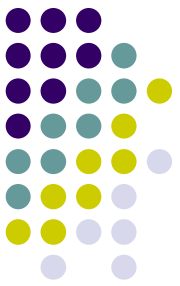




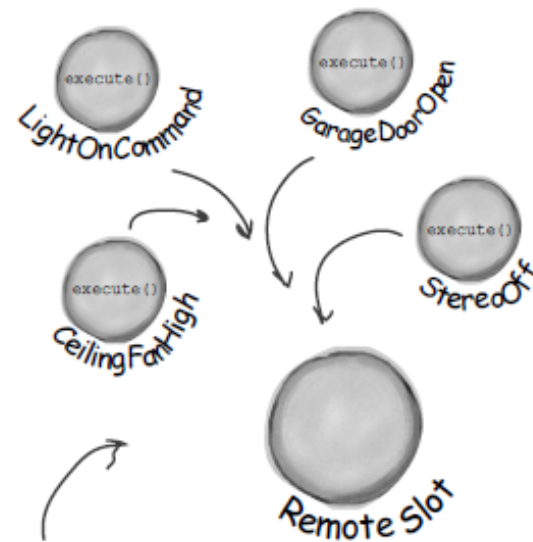
Command

- Why use command objects?
 - **Easier to construct** general components that need to delegate, sequence or execute method calls
 - **No need to know** the class of the method or the method parameters.
 - Using an **invoker object allows bookkeeping** about command executions to be conveniently performed
 - implementing different modes for commands
 - managed by the invoker object

Command



An encapsulated request.



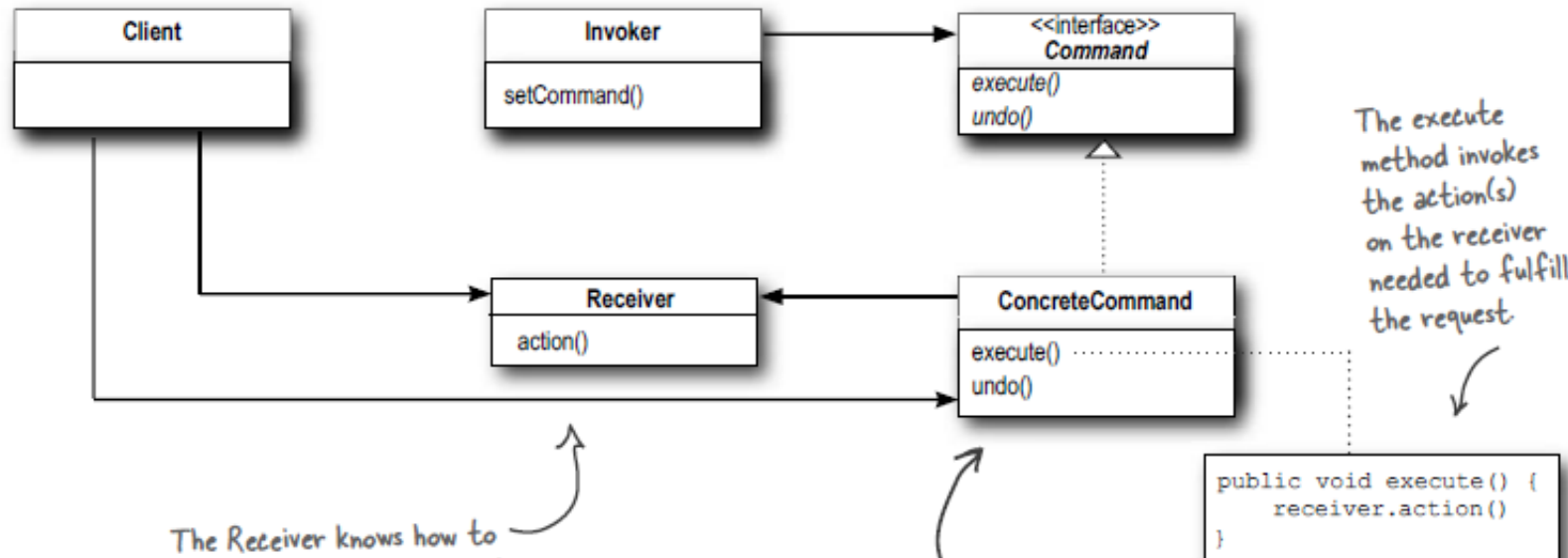
An invoker – for instance
one slot of the remote
– can be parameterized with
different requests.

Command

The Client is responsible for creating a ConcreteCommand and setting its Receiver.

The Invoker holds a command and at some point asks the command to carry out a request by calling its execute() method.

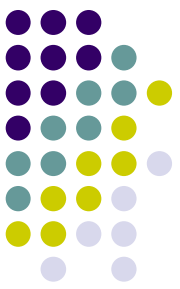
Command declares an interface for all commands. As you already know, a command is invoked through its execute() method, which asks a receiver to perform an action. You'll also notice this interface has an undo() method, which we'll cover a bit later in the chapter.



The Receiver knows how to perform the work needed to carry out the request. Any class can act as a Receiver.

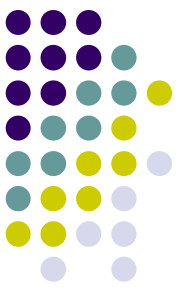
The **ConcreteCommand** defines a binding between an action and a **Receiver**. The **Invoker** makes a request by calling `execute()` and the **ConcreteCommand** carries it out by calling one or more actions on the **Receiver**.

The `execute` method invokes the action(s) on the receiver needed to fulfill the request.



Command in Game Engine

- Where do you deal with Command Objects?
 - Instead of using delegates or functors
 - Language agnostic
 - Customizable method calls
 - Unified way to deal with various types of commands
 - Sequence of commands
 - Create a macro of commands

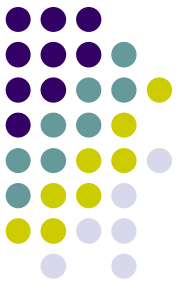


Design Patterns

- Easy tutorial book on design patterns
 - Get your copy today
 - Ecopy at Depaul Library through the safari link



Thank You!



- Questions?

